

Library Management System

Project Overview

The Library Management System is a full-stack web application developed using Python, Flask, MySQL, HTML, CSS, and JavaScript. The system helps libraries manage books, members, borrowing, returns, and fines efficiently through role-based access for Admins, Librarians, and Members.

Features

Authentication & Authorization

- User Login and Logout
- Role-Based Access Control
- Admin
- Librarian
- Member
- Session Management

Book Management

- Add New Books
- View Books
- Update Book Details
- Delete Books
- Search Books by Title, Author, or Genre
- Track Available Copies

Member Management

- Register Members
- View Member Details
- Update Member Information
- Deactivate Members
- View Borrow History

Borrow & Return Management

- Issue Books to Members
- Return Books
- Automatic Due Date Calculation (14 Days)
- Prevent Borrowing if No Copies Available
- Prevent Borrowing if Unpaid Fines Exist

Fine Management

- Calculate Fine Automatically
- Rs. 5 Fine Per Overdue Day
- View Fine Records
- Mark Fines as Paid

Dashboard

- Total Books
 - Total Members
 - Active Borrows
 - Overdue Books
 - Total Fines Collected
-

Technologies Used

Backend

- Python
- Flask
- MySQL

Frontend

- HTML
- CSS
- JavaScript

Tools

- Git
 - GitHub
 - Postman
 - VS Code
-

Project Structure

library-management-system/

|— app.py

|— config.py

|— routes/

| |—— auth.py
| |—— books.py
| |—— members.py
| |—— borrows.py
| |—— fines.py
|—— static/
| |—— css/
| |—— js/
| |—— pages/
|—— templates/
|—— README.md

Database Setup

Create Database

```
CREATE DATABASE library_ms;  
USE library_ms;
```

Create Tables

Tables used:

- users
- books
- members
- borrows
- fines

(Use the SQL schema provided in the project requirements.)

Installation Steps

1. Clone Repository

```
git clone <repository-link>  
cd library-management-system
```

2. Create Virtual Environment

```
python -m venv venv
```

3. Activate Environment

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

4. Install Dependencies

```
pip install -r requirements.txt
```

5. Configure Environment Variables

Create a `.env` file:

```
DB_HOST=localhost  
DB_USER=root  
DB_PASSWORD=your_password  
DB_NAME=library_ms  
  
SECRET_KEY=library_secret_key
```

6. Run Application

```
python app.py
```

Server will run at:

```
http://127.0.0.1:5000
```

API Endpoints

Authentication

Method	Endpoint	Description
POST	/register	Register User
POST	/login	Login
POST	/logout	Logout

Books

Method	Endpoint
GET	/books
POST	/books
PUT	/books/<id>
DELETE	/books/<id>
GET	/books/search

Members

Method	Endpoint
GET	/members
POST	/members
PUT	/members/<id>
GET	/members/<id>/history

Borrows

Method	Endpoint
POST	/borrow
POST	/return/<borrow_id>
GET	/borrows/active
GET	/borrows/overdue

Fines

Method	Endpoint
GET	/fines
POST	/fines/<id>/pay

Team Contribution

Module 1

Database & Authentication

Module 2

Book Management Backend

Module 3

Member Management Backend

Module 4

Borrow, Return & Fine Management

Module 5

Admin & Librarian Dashboard

Module 6

Member Portal & QA Testing

Challenges Faced

- GitHub merge conflicts during integration.
 - Connecting frontend pages with backend APIs.
 - Managing role-based authentication.
 - Fine calculation and borrow-return workflow.
-

Learning Outcomes

- Flask API Development
 - MySQL Database Integration
 - Frontend and Backend Integration
 - GitHub Collaboration
 - REST API Testing using Postman
 - Team-Based Software Development
-

Future Enhancements

- Book Reservation System
 - Export Reports as CSV
 - Pagination
 - Book Cover Images
 - Email Notifications
 - Online Fine Payment
-

Conclusion

The Library Management System successfully manages books, members, borrowing, returns, and fines through a user-friendly interface and secure role-based access. This project provided hands-on experience in full-stack web development, API integration, database management, and collaborative software development.